

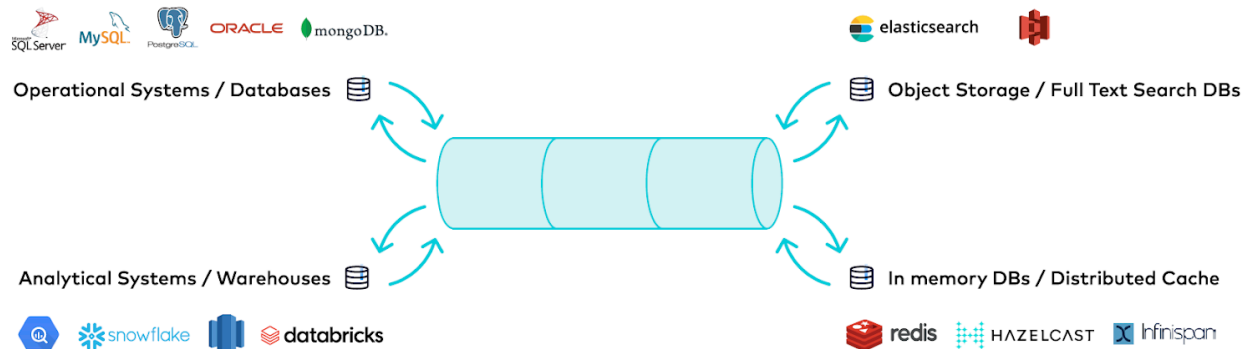
Streaming Database Ingestion: A Shift-Left Approach



Table of Contents

Introduction	2
Solution	3
Sourcing from a Database (Extract)	4
Poll-Based CDC	5
Log-Based CDC	6
Sinking to a Database (Load)	7
Transform Data (Transform)	7
Connector-Based Stateless Transformations	7
Streaming Stateful & Stateless Transformations	8
Use Cases	8
Migrate/Replicate Between Source and Target Systems (1:1)	9
Materialized Denormalized View at Source Database	10
Streaming Enrichment	10
Denormalize at the Target Storage Engine (ELT)	11
Outbox Pattern	11
Microservices Outbox Pattern	11
Replace XA Transactions for Multiple Database Writes	12
Considerations	12
Security and Data Privacy	12
Data Governance	13
Performance and Scalability	13
Monitoring and Reliability	14
Getting Assistance	15
Resources	15
Contributors	15

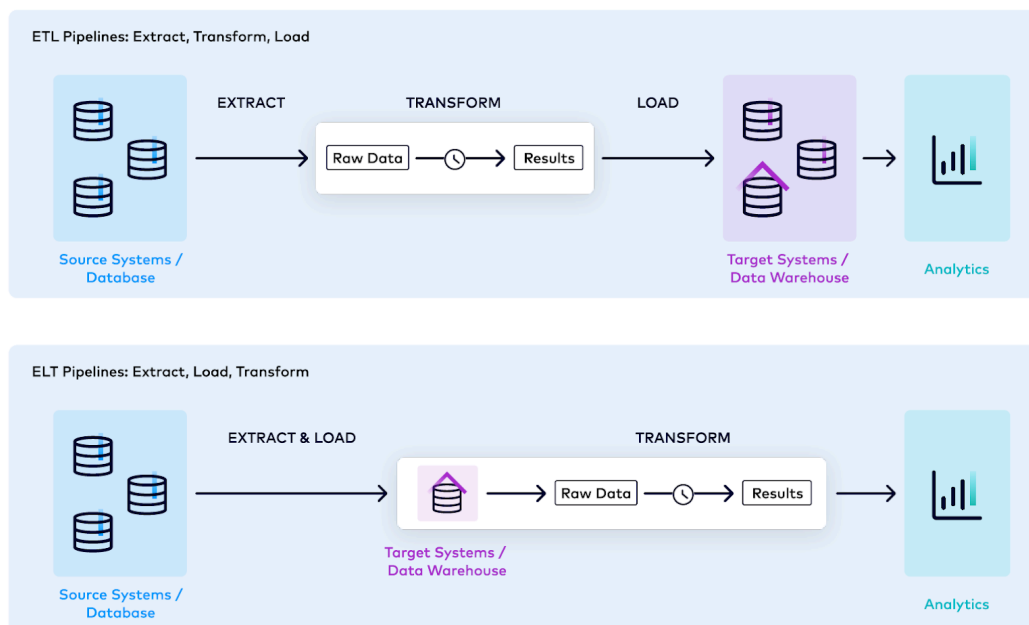
Introduction

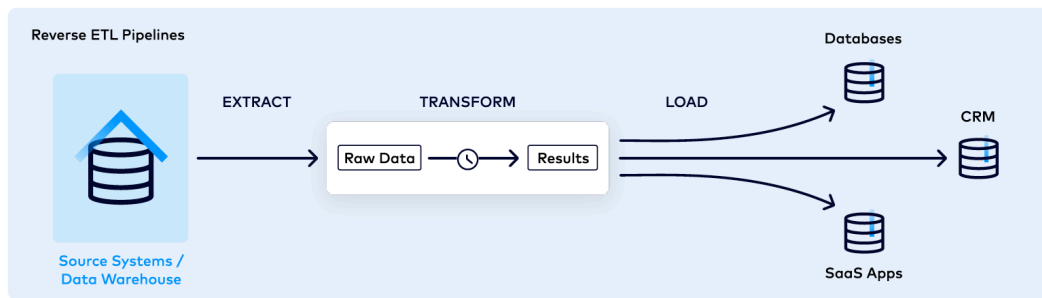


Organizations store data in multiple systems and databases. The process of moving data from one system to another is referred to as a “data pipeline.” Data pipelines move data in an organization between disparate operational systems and analytical systems. Data pipelines can be set up as batch-based pipelines or as continuous streaming pipelines.

A pipeline facilitates the extraction (E) of data from a source storage engine, applies one or more stateless/stateful transformations (T) to prepare the data for loading (L) into sink targets. The standard patterns of data pipelines are as follows:

- **ETL:** Data is extracted from a source system or database, transformed in-process, and loaded into a target system or data warehouse.
- **ELT:** Data is extracted from a source system and loaded as-is into a working or staging area of the target system. A process then transforms it within the target system.
- **Reverse ETL:** Data is extracted from a data warehouse, or other system which might be the target of a standard ETL process, is transformed and loaded into operational systems.



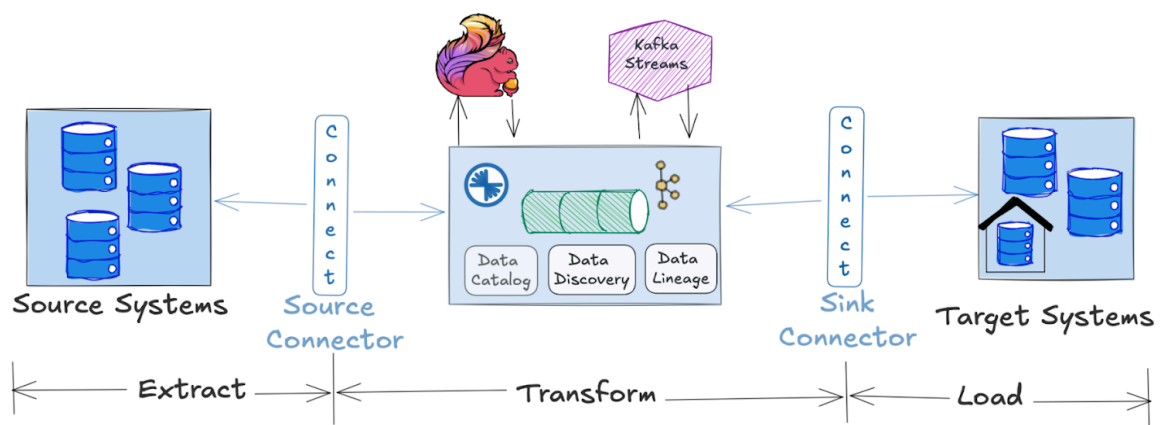


Modern cloud applications require access to multiple databases in real time. However, integrating data fragmented across silos of distributed databases for a complete, up-to-date view is complex and expensive. Existing point-to-point pipelines are rigid and brittle with data fidelity and governance challenges. ETL, ELT, and reverse ETL processes are built around batch processing and batch data delivery that result in outdated snapshots and stale information, which are unsuitable for real-time use cases.

With Confluent, teams can shift-left to the Data Streaming Platform to connect, stream, process, govern, and share curated data streams in real-time enabling self-services access to the right data, in the right format, anywhere needed.

This white paper focuses on building streaming pipelines between different Databases (RDBMS).

Solution



“Kafka Connect Framework” allows organizations to integrate Apache Kafka with various tech stack and storage engines. One can self-manage Kafka Connect, but Confluent Cloud offers a fully managed connect framework as well as a broad selection of fully managed connectors and the ability to upload your own custom connector, lightening most of the operational burden behind Kafka Connect. Confluent provides a **rich portfolio of connectors** built by Kafka experts that provide enterprise-grade security, reliability, compatibility and support. Confluent connectors enable organizations to build streaming pipelines to databases even faster with powerful integrations (just by configuration with no new code). There are two types of connectors:

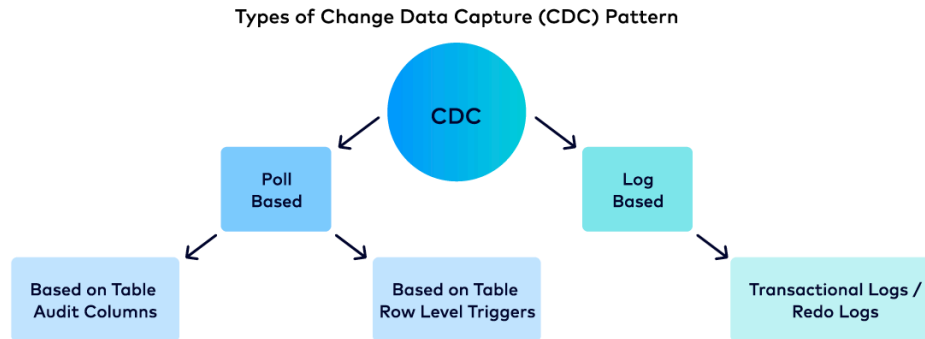
- **Source Connectors** move data between a Storage Engine or Source Systems, to one or many Apache Kafka topics. Source Connectors have a built-in producer to write data to Kafka.

- **Sink Connectors** move data from Kafka topic to Storage Engine or Sink Systems. Sink Connectors have a built-in consumer to consume/read data from Kafka topics.

Sourcing from a Database (Extract)

During the Extract phase, data is extracted from one or more sources or systems.

Change Data Capture (CDC) is a technique to capture/track database changes as they happen, and generate a continuous stream of row-level changes. The changes are emitted as events in the same order as they were committed to the database and in near real time.



Depending on application requirements and tolerance for performance overhead, the process of capturing the changed data in a table is broadly classified into two categories:

Poll-Based CDC	Log-Based CDC
Overview: In this type of CDC, the Database table's data are extracted by periodically executing a SQL query (at configured intervals) and the changes are emitted as events. Cons: • Constant polling on DB tables adds pressure on the DB and may degrade DB performance.	Overview: Log-based CDC makes use of the DB's internal mechanism of transactional logs. Database implements "Transaction logs/change logs" to record all Events and also as a mechanism to recover (in the event of any failure). In a log-based CDC, a process continuously parses transaction logs and emits event records. Most of the DB implementations provide some kind of an API to read the change logs. Cons: • Most DBs do not document the internal format of transaction logs. Every DB's internal representation of txn logs is different, and parsing the internal format of transaction log format is complex. • Every release internal logic may change.
Implementation Types: Audit Columns: • To track updates between polling intervals, identify or add a column (Incrementing column or a	Implementation Types: Transactional Logs/Redo logs: • Changes are captured without application-level modifications (or) without having to scan operational

Timestamp column) to the table.

- Changes can be extracted using SQL query.
- Supports bulk mode, SQL projections.

Cons:

- Multiple updates between polling intervals are not tracked or lost.
- Unless it's a soft delete, Deletes are not propagated to target systems.
- History of DDL may not be tracked.

Row-Level Triggers:

- Enable Row-Level Triggers (before or after) for every Insert, Update or Delete in a Table to a shadow table on the same database.
- Shadow tables represent immutable, detailed logs of all transactions.
- Deletes and updates are tracked. Uses native SQL API for DBs.

Cons:

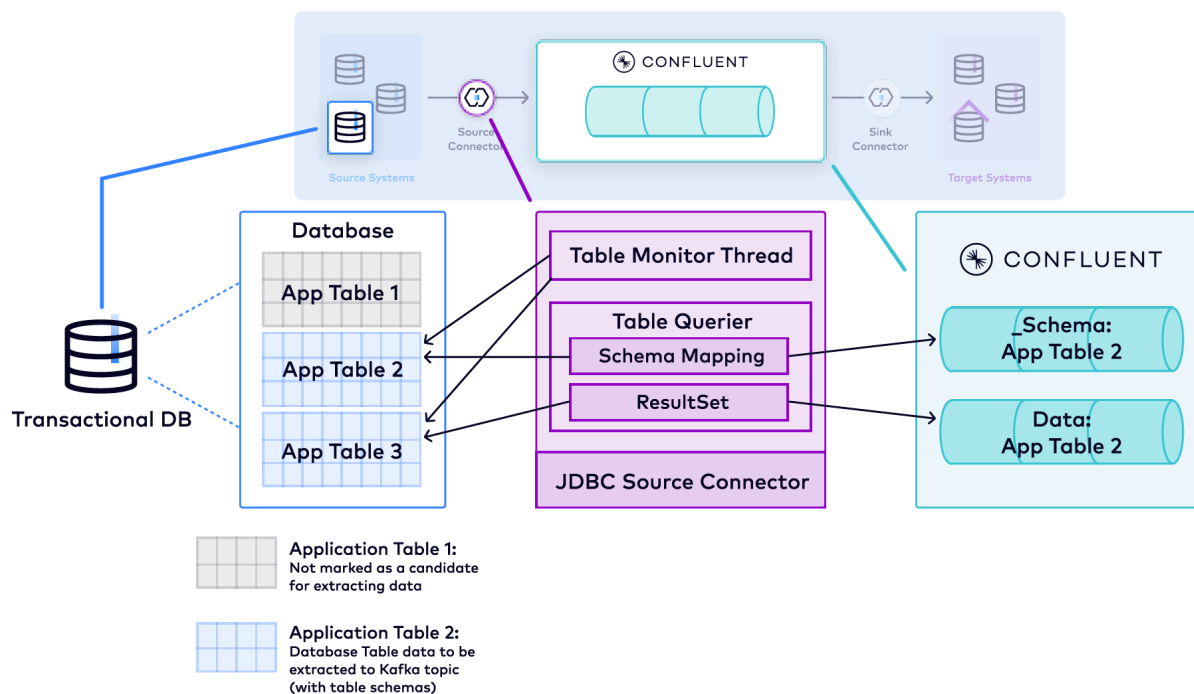
- Since triggers are on a Row, the performance of the DB is significantly impacted.

tables.

- Does not impact the source DB's performance.
- Can maintain ACID reliability.
- No requirement to modify the production DB table schemas (or) add additional tables.
- Minimal impact to production DB system.
- Update, Delete, and Schema changes to DB tables (Metadata) are all tracked.
- Debezium is one of the popular open source CDC connectors for various databases like PostgreSQL, MySQL, SQL Server, etc.

Progress in row-level table processing is monitored on a Kafka Connect topic (connect_offsets). This tracking is crucial for connector restarts. Most CDC-based connectors offer "at least once" delivery guarantees.

Poll-Based CDC



Confluent JDBC Source Connector is an example of a poll-based CDC. Inside the JDBC Source Connector process:

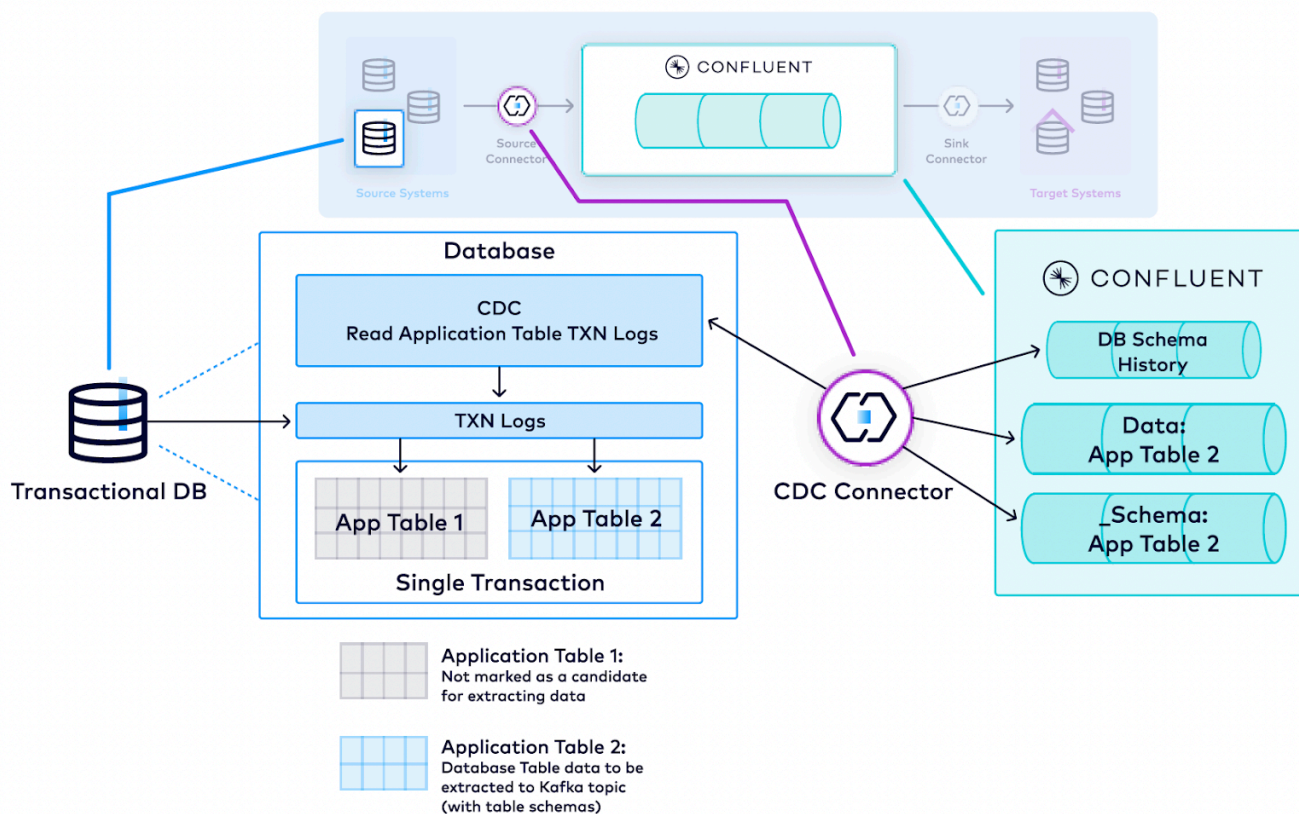
- **A Table Monitor Thread** polls the configured tables for every configured interval and identifies the number of tasks to run. The thread also monitors for any new tables created or deleted tables.
- **TableQuerier** — Each Task has a number of TableQuerier in a Queue. Each TableQuerier executes the configured SQL, reads the next row, and converts the row to a ConnectRecord. It also makes an instance of Schema Mapping (per table) and uses it to map a row into a SchemaAndValue object.
- If **Schema Registry** is enabled on the connector, then the structure/schema of the record written to the Kafka topic (Application Table data) along with schema version changes (if any) are tracked in `_schema` topic.
- **Table's row data** lands in a data topic and the structure/schema of the table are tracked in `_schema` topic.

By default, there is a 1:1 relationship between Database tables and Kafka topics. For each table in the database, the connector creates a Kafka topic. SMTs can be enabled to route data to different topics. (Check out the [Transform](#) section for any denormalization requirements.)

Note: Confluent's Premium Oracle CDC Connector has both the variant, version 19 and below uses continuous mining (CDC). Since continuous mining is deprecated on versions above 19, the connector works as a poll-based connector but it polls on LogMiner logs instead of the main table.

Log-Based CDC

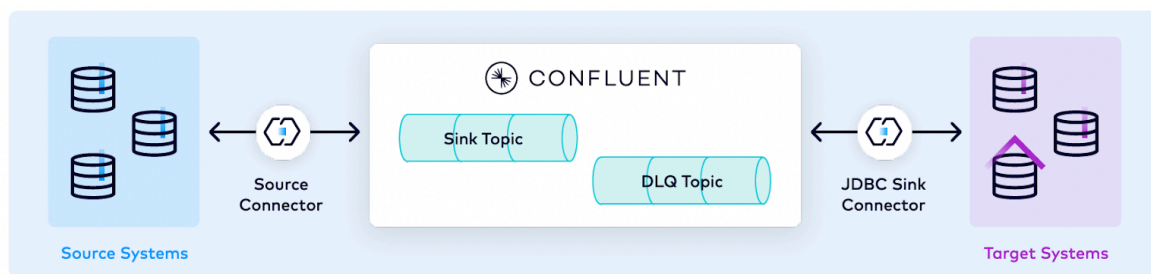
In a high-velocity data environment or in environments with time-sensitive data, log-based CDC is a great fit to achieve reliable, scalable, and low-latency data replication. Log-based CDC is also ideal for zero-downtime migrations to the cloud.



Database has transaction logs, and a CDC connector reads the database transaction logs on the Source database and emits changes as events. Appropriate privileges/grants must be granted for the CDC process to read transaction logs. The CDC process reads the transaction logs for various Data Definition language (DDL) statements (Schema changes) and Data Manipulation language (DML) statements (table row data) and generates events (in the order it reads the transaction logs).

- **DB Schema history:** An internal topic only used by CDC Debezium connector to track DB table schema history.
- **Data topic:** By default there is a 1:1 relationship between DB table and generated Kafka topic.
- **_Schema:** The structure of the data written to the data topic (App Table 2) is tracked in _schema topic.

Sinking to a Database (Load)



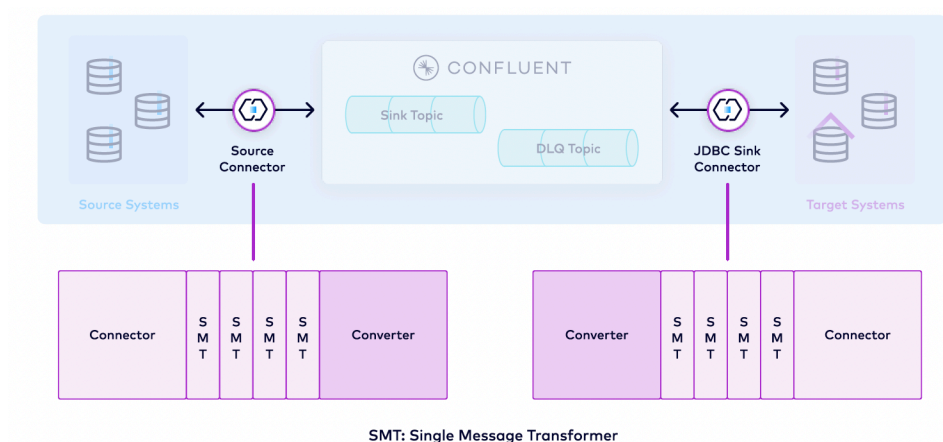
- Sink Connector internally has a consumer which can consume data from topic and write to destination storage engines. The progress of the record synced to a storage engine is either stored in __consumer_offset topic in Kafka or on the storage engines (if the storage engine supports it).
- SMT can be enabled on Sink Connectors to apply any stateless transformations.
- Multiple Sink Connectors can be run in parallel to sink Kafka topic data to multiple storage systems.
- JDBC Sink Connector also supports DLQ topics to handle any error scenarios.
- Most Sink Connectors support “at-least once” guarantees. Some of them support idempotent writes (using upserts). There are a few Sink Connectors which also support the “exactly once” delivery guarantees (e.g., S3 Sink, Azure Blob Storage Sink, Google Cloud Storage Sink, HDFS 2 Sink).

Transform Data (Transform)

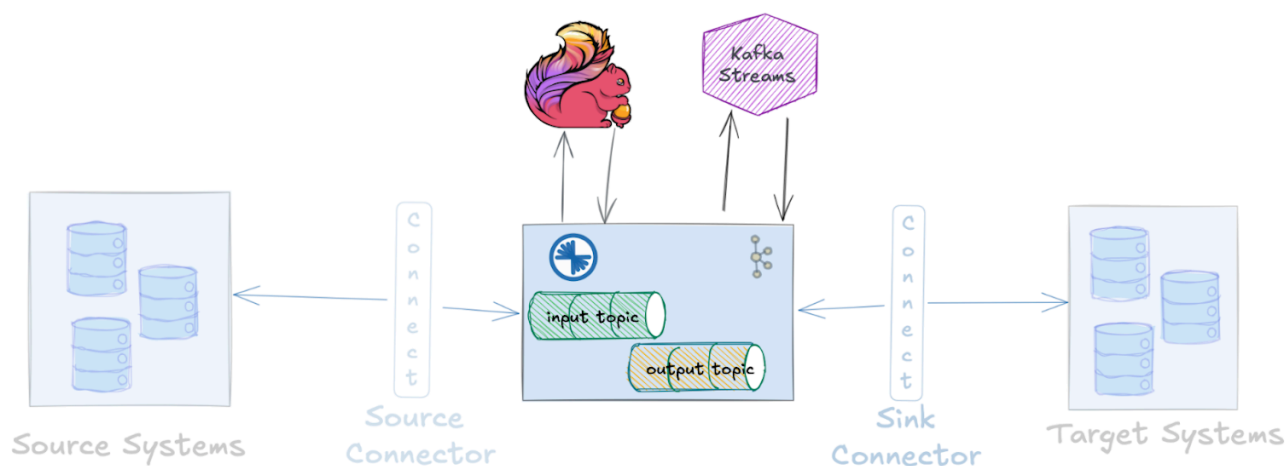
Connector-Based Stateless Transformations

For scenarios where a record has to be transformed “in-flight”, Connect Framework supports SMTs (“Single Message Transform”) to transform records at both Source and Sink Connectors.

- **Connector:** SourceConnector connects to a Source Storage Engine, reads data, and converts it to a Canonical ConnectRecord Format to be used by SMTs. SinkConnector gets a ConnectRecord format data from SMT and Writes to Target Storage Engines.
- **Converter:** SourceConverter converts ConnectRecord input to a Serializable format (AVRO, JSON, PROTOBUF) to store data in Kafka. SinkConverter deserializes Data from a Kafka Record and converts it to ConnectRecord format.
- **SMT:** One or many SMTs can be chained together to do a stateless transformation in the order declared in JSON connector configuration. For each SMT, input is in ConnectRecord format and output is also in ConnectRecord format. SMTs are an Anti-pattern and not to be used for stateful transformations or stateful lookups (e.g., Rest API call or DB lookup).



Streaming Stateful & Stateless Transformations



- Once data lands in the Kafka topics, Apache Flink or KafkaStream Application can be deployed to do Stateless or Stateful transformations on the Dataset.
- The data in kafka topics can be sourced from different Databases or data sources. Join multiple topics data in near real time to enrich or to transform data to output topics.
- Kafka Streams provides KTable and GlobalKTable abstractions, Apache Flink provides Table constructs to do a lookup and/or joins of topic data in near real time, and write the consolidated transformed data to an output topic.
- In this type of transformation, Apache Flink or Kafka Streams application reads data from an input topic (from a bootstrapped Kafka server) and writes to an output topic in the same Kafka server (with enriched data).
- Lookup data for enrichment can be sourced as a topic from CDC Source Connectors.

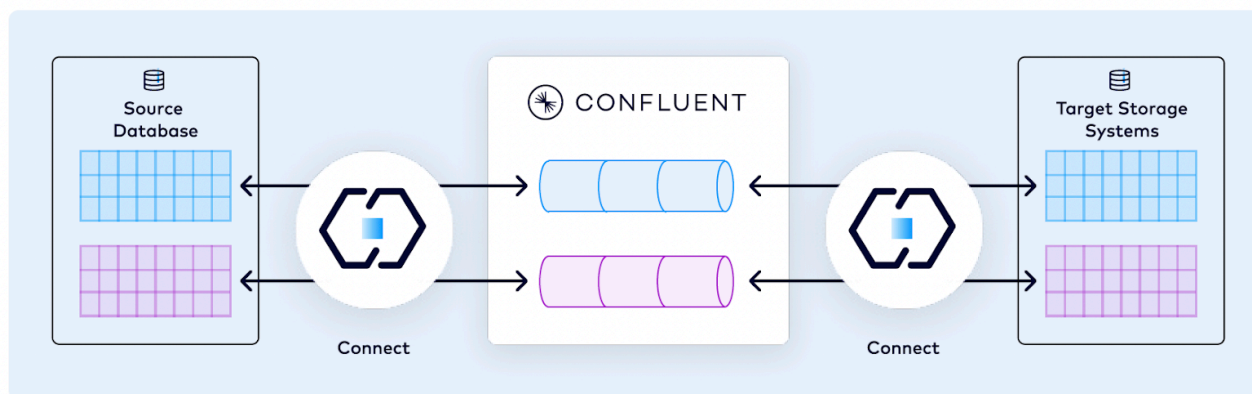
Use Cases

In an organization, when a connector framework is integrated with Kafka and various storage engines, efficient pipelines are created for data movement between source and sink targets. Some data pipeline use cases include:

- Application or Database migration. Synchronizing data between multiple database systems.
- Keeping your Cache layer in sync with the Database. Updating Search Indexes.
- Accelerating business intelligence & reporting needs.
- Pushing data to multiple lines of business.
- Hub & spoke model to aggregate and improve your MDM systems.
- Integrating applications with otherwise incompatible Databases (also used in microservices). Outbox pattern.
- Reducing pressure on operational databases.
- Disaster recovery or backup plan.
- Building Reactive applications based on changes to the database.

Kafka Connect framework implementation patterns for some of the popular use cases are given below.

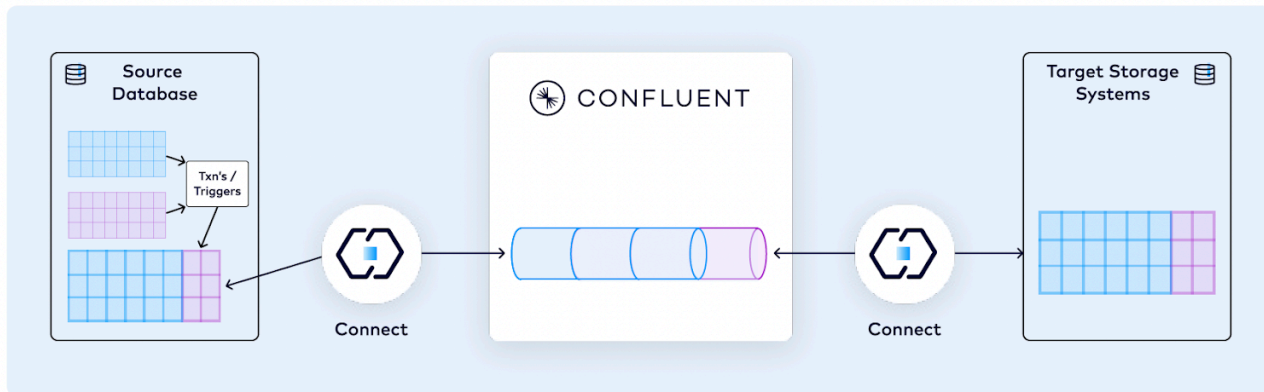
Migrate/Replicate Between Source and Target Systems (1:1)



Single Table to a Single Topic

- Used during Disaster Recovery, Database migration/replication.
- Copy each source table to a topic in Kafka.
- 1:1 mapping of table and topic.
- Enable lightweight stateless transformation using SMT at Source and Sink Connectors.
- Copy table data from a source system to a topic on Kafka and subsequently to a table on the destination database or target system (all in near real-time).

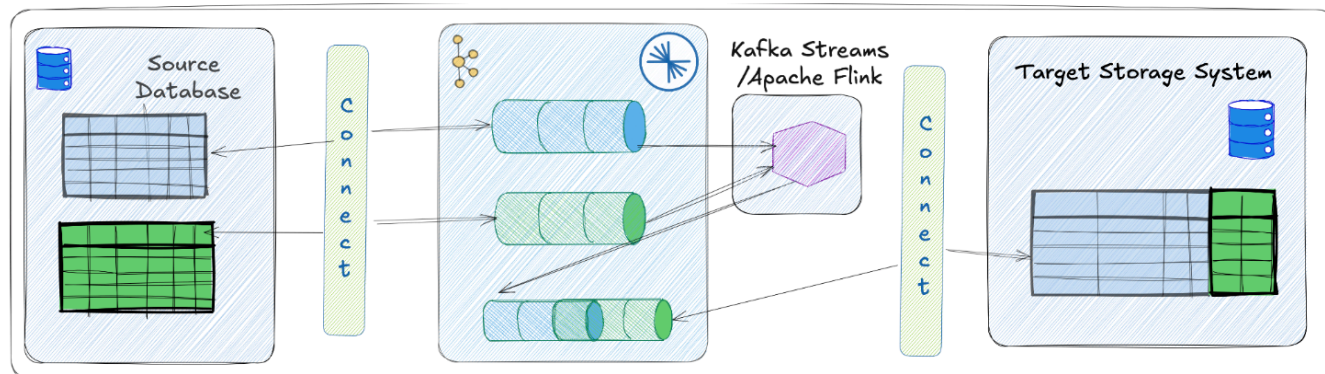
Materialized Denormalized View at Source Database



Stage or Prepare Materialized Denormalized Table View at Source Database

- Works well in scenarios of FanOut, where a copy of “Source of Truth” is needed at multiple storage engines (RDBMS data copied to “cache,” text search engine Elasticsearch, document database MongoDB, etc.).
- Prepare or prep up the data on the source databases.
- Join the data in one or more tables on the source DB, which will then be loaded via CDC.
- Data integrity is checked on the source system.
- Sink topic data to target systems.

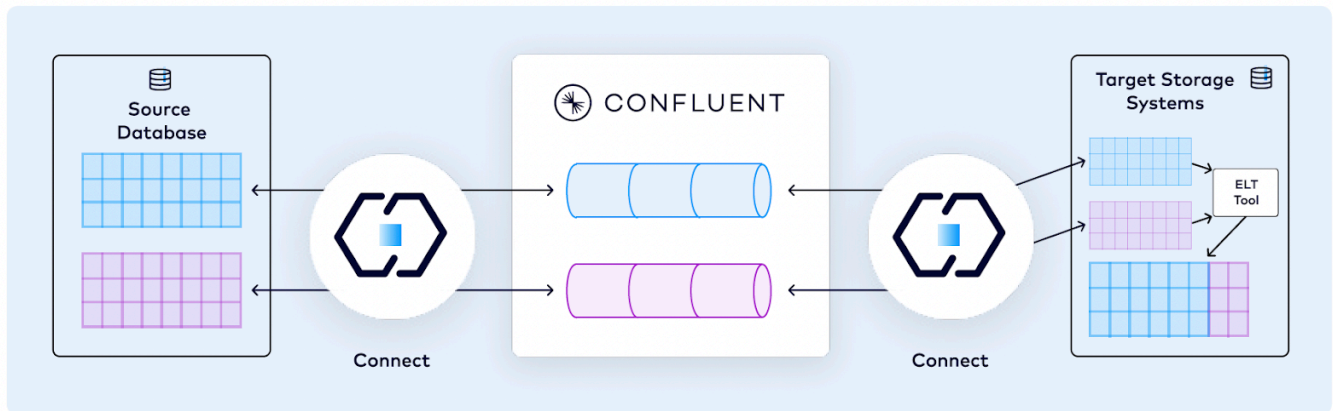
Streaming Enrichment



Single Table to a Single Topic. Microservice enriching & creating a denormalized view

- Works well in time-sensitive joins (where a network hop to a database for lookup or joins may add latency to the stream processing).
- Extract necessary data as-is from source systems to Kafka topic, and extract lookup data to Kafka topics.
- Apply stateless or stateful streaming enrichment on the topics. (Join multiple topics' data to a single output topic.)
- Use the Sink Connector to connect to multiple sink storage engines.

Denormalize at the Target Storage Engine (ELT)



Denormalize at the Destination Using ELT Tool

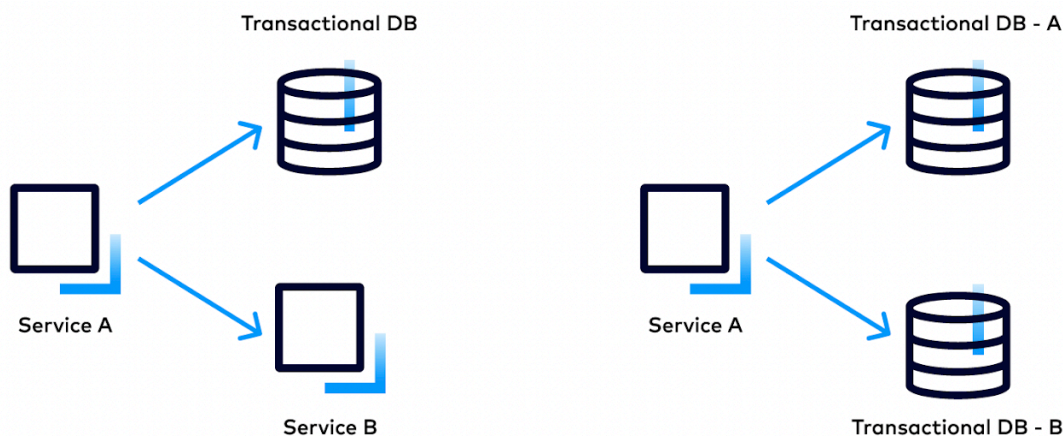
- ELT scenario at target systems.

Outbox Pattern

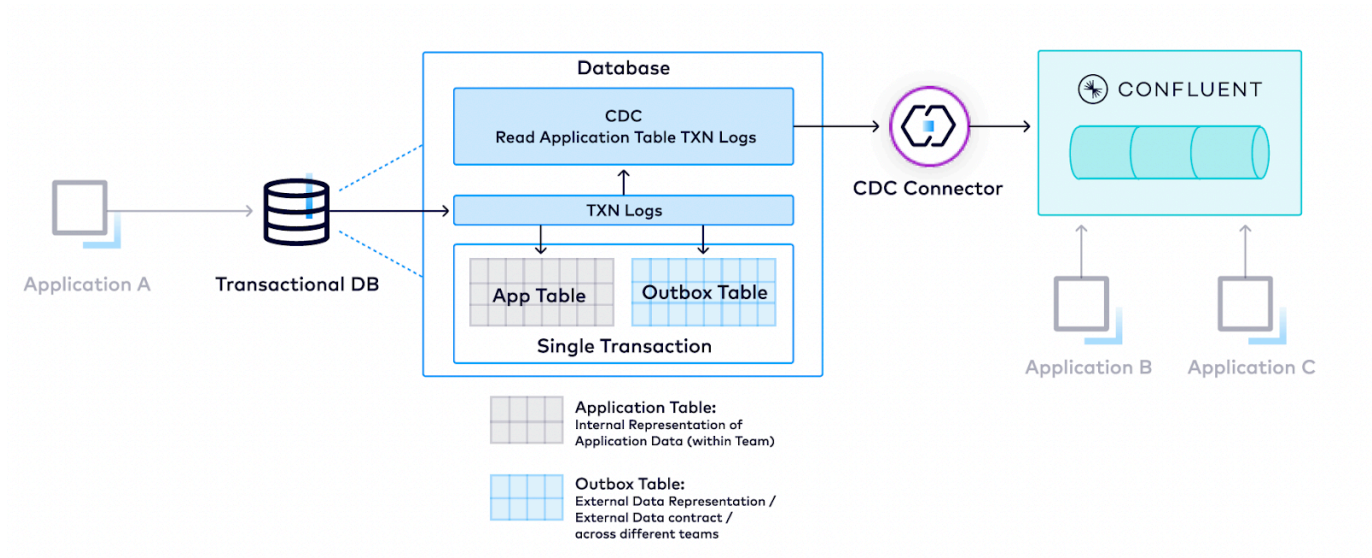
Microservices Outbox Pattern

Problem Space: In a Microservices environment, there is always a need for one application/microservice to update a DB and also invoke another service within a single Transaction boundary.

Solution Space: Application writes data to its table in the database and also to an outbox table within a single Transaction. The outbox table is sourced to Kafka topics.



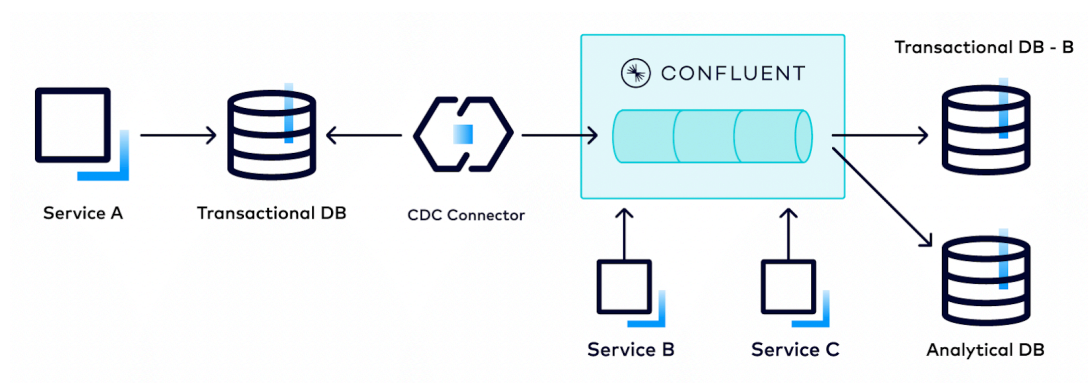
Scenario: In DDD systems an Aggregate's internal representation of data may/should be different from the outside contract. In that scenario, the outbox pattern can be applied.



Replace XA Transactions for Multiple Database Writes

Problem Space: One Application needs to write to two different database storage engines where XA transactions are not supported.

Solution Space: Instead of XA transaction, the Application writes data to a single database table, which is sourced to a Kafka topic by a Source Connector. Multiple Sink Connectors can be used to sink data to different storage engines.



Considerations

Security and Data Privacy

- **Access:** Apply the Principle of least privilege (limit access to source and target RDBMS, limit access to Kafka using ACLs or RBAC). Check firewall clearance. Check Licensing constraints (e.g., Oracle GoldenGate license).

- **Roles:** Create a DB user to connect to the DB. On source DBs, grant appropriate privileges to the DB user to access the DB transaction logs. Configure appropriate database transaction log levels (e.g., setting correct `wal_level` for Postgres, enabling Archivelog mode and supplemental logging for Oracle). Use Row Level Security on the RDBMS side (JDBC Connector). Consult with the DBA team to size the transaction logs properly.
- **Handling PII Data:** Consider Anonymizing, masking or sanitizing data using SMT in Source & Sink Connectors before it hits the disk.
- **Data Encryption:** Encrypt data (on wire and at rest). Establish an end-to-end security policy (e.g., Who is allowed to access data in Kafka and RDBMS?). Manage rotation of credentials, revoking, etc.

Data Governance

- **Data Catalogs:** Data catalogs help in the discoverability of data. In Confluent Cloud, Data catalog feature is enabled for topics associated with the Schema Registry. Topics and Schema can also be tagged with additional metadata (classification/tagging the data). Confluent Cloud provides a global search feature for data discoverability by indexing the metadata, so that any topic, schema, tag, or connector can be easily discovered from global search. Confluent Cloud's Schema Registry and tags are the source for data catalogs to display the types of data flowing through topic/pipes.
- **Data Contracts:** Data ingested into Kafka topic from Source Connectors may include schema in raw or unrefined format (e.g., Debezium Source Connectors with attributes such as "before," "after," "source," "operation," etc. are considered to be in Raw format). This data once processed by the streaming engine is written to a secondary topic (refined/schema data) and tagged appropriately as "refined data." Enable Schema Registry to enable data contracts and tag appropriately to differentiate between raw and refined data.
- **Data Validation:** Consider enabling Schema Registry with connectors (Source and Sink Connectors) early in the project, so that any data written to a topic (Source Connector) or read from a topic (Sink Connector) is validated for appropriate schema. Enable Compatibility mode on SR to validate any version change issues.
- **Data Lineage:** Data lineage in Confluent Cloud is visually represented as a lineage graph. Stream Lineage shows the movements of data from source to destination along with any transformation applied.

Performance and Scalability

- **Check Sizing Requirements:** Consider involving a DBA to properly size the source and target systems, transaction logs (Redo, WAL), memory, and disk. Set up the Connect framework to run in distributed mode for horizontally scaling worker nodes.
 - **For a Source Connector:** Even though 1 task can read data from many tables in source DB, for maximum performance consider increasing the total number of tasks to match the total number of DB tables (e.g., if there are 5 tables to be sourced, then the maximum number of tasks that could be run is 5). For managed connectors, in a few scenarios, multiple connectors may have to be run in parallel (e.g., 1 table, 1 task, 1 connector).
 - **For Sink Connectors:** The maximum number of tasks depends on the total number of partitions considered for data migration.
- **Guaranteeing Capacity:** Consider enabling Quotas/resource limits to ensure capacity is guaranteed.
- **Latency Considerations:** Consider eliminating bottlenecks on the Network, disk, and CPU. And for tuning end-to-end latency on pipelines, check if there are any lags between Kafka topics and storage engines. Based on your requirement, consider the following.
 - **For Source and Sink Connectors:** Consider increasing the number of tasks to parallelly read from multiple tables (source) or to read from multiple partitions (for sink).
 - **For topics:** Size your Kafka topic with required partition count to reduce the latency in the pipeline.

- **For Source Connector:** if possible, consider Reading from standby DB (replica) to ease the impact of mining transaction logs.

Monitoring and Reliability

- **High Availability:** For Databases, consider implementing standby replicas (read replicas) on the DB side. On source and target databases:
 - Use the built-in HA features like Transaction Log Mirroring.
 - Use the near-zero downtime features of your platform.
 - Oracle RAC and/or DataGuard
 - Postgres replication or Patroni
 - SQL Server always-on cluster
 - Kafka connect High Availability: Setup connect cluster in “distributed mode.”
 - Consider involving DB team and Network team early in the project during HA setup. HA setup mitigates single point of failure (SPOF) or complete data loss.
- **Disaster Recovery:** Early in the project, define and document Recovery Point Objective (RPO) and Recovery Time Objective (RTO) requirements. Implement mechanisms on the database to facilitate the promotion of standby replicas to primary. Disaster Recovery (DR) for a connector is contingent on several factors; a connector should only fail over to another region if the database in the DR region is also available and configured for DR strategies. Few considerations for connector are:
 - **Source Connector:**
 - Where does the connector track the progress of reading data from storage systems (e.g binlogs status, or in Kafka topics connect_offset).
 - If the Source Connector’s Kafka topics are replicated to DR Kafka cluster.
 - Is it possible to access the connector metadata topics (e.g., self-managed connector has access to connect_configs, connect_status & connect_offsets) so that connect_offsets may be updated during DR (for fully managed connectors these internal topics are hidden).
 - If the upstream DB replica is not accessible, can the connector read from a read replica (e.g., MySQL if GTID replication can be enabled, Source Connector can read from another read replica during restarts).
 - On the DR region, depending on the scenario, if the Source Connector cannot be started from a specific point in time or position, should another snapshot be created or only the change data needs to be sourced to a Kafka topic.
 - **Sink Connector:**
 - When configuring Sink Connectors with an input topic replicated from a primary cluster, manage offset settings carefully. If set to “earliest,” duplicate data may be processed. Use “kafka-consumer-groups” to reset offsets and prevent this.

Monitoring is a crucial task to ensure the availability and stability of the system. Here are some considerations to keep in mind:

- **Metrics:** Define the key indicators that you want to track. For example, response time, error rate, throughput, memory usage, CPU usage, and network latency.
- **Data Collection:** Decide what data you need to collect to calculate the metrics you defined and how to extract that data (e.g., jmxexporter, Datadog). Identify where to store that data (e.g., Prometheus).
- **Visualization:** Use dashboards to visualize the data collected by the monitoring tools (e.g., Grafana, Datadog).
- **Alerting:** Define the thresholds for each metric that will trigger an alert. For example, if the error rate exceeds 5%, send an alert to the operations team.

- **Maintenance:** Regularly review and maintain the monitoring setup to ensure it remains up-to-date with any changes to the application and infrastructure.
- **Security:** Ensure the monitoring tools and data are secure, and access is restricted to authorized personnel only.

Getting Assistance

This white paper provides an overview of implementing a specific use case in a generic manner. However, every situation is unique and enterprise needs often require nuanced solutions. [Confluent's professional services team](#) will expertly guide you in making the right technical decisions, accelerating a successful build, and unlocking the full potential of real-time data within your organization. [Contact us](#) to take advantage of our extensive experience.

Resources

- [Try Confluent Cloud for free](#) and leverage 120+ pre-built connectors to build streaming pipelines across your data architecture.
- Watch a hands-on [Show Me How for building streaming pipelines to cloud databases](#).
- GitHub repo: [Stream Data to Cloud Databases](#).
- Visit the [Shift-Left Analytics](#) solution page to learn more.
- [Contact Confluent](#) to discuss your particular use case.

Contributors

Amith Manjunath